MongoDB 8.0 · INDICES PRACTICAS

Indexación

Cuaderno de laboratorios

Fichas con órdenes, pruebas negativas, explicaciones causales y espacio para resultados.

Entorno del curso: el repositorio MongoDB contiene `compose.yaml` y los scripts de `lab/`. Ejecuta los pasos en el orden indicado, prepara los datos necesarios y contrasta lo que observas con el resultado explicado.

Antes del primer laboratorio

Mapa de ejecución sin improvisaciones

Equipamiento y entorno

- Equipo de 8 GB RAM como referencia, Docker Desktop activo, terminal PowerShell o Bash, espacio libre suficiente para los volúmenes y conexión para descargar mongo:8.0.
- MongoDB Community 8.0 y mongosh vienen dentro de las imágenes; Compass es opcional. No se exige licencia Enterprise para los ejercicios esenciales.

Preparación del formador

```
Copy-Item .env.example .env
# Edita .env antes de iniciar los servicios.
docker compose up -d --wait mongo-indices mongo-seguridad
docker compose ps
```

Dos contextos distintos

La terminal del sistema ejecuta docker compose; mongosh ejecuta consultas y administración MongoDB. Si una instrucción inicia con use curso_indices o db., se escribe dentro del shell de MongoDB, nunca en PowerShell.

Normas de reproducibilidad

Los scripts del paquete contienen datos de laboratorio exclusivamente. Las operaciones que borran ventas o colecciones demo están identificadas. Al repetir una carga, se pierden los índices secundarios de la colección reinicializada. No ejecutarlos sobre servidores externos ni entornos con datos propios.

PREPARAR → EJECUTAR → OBSERVAR → EXPLICAR

Ruta esencial de tres horas

Indexación: carga, línea base, índice simple, comparación, compuesto ESR, text y TTL. Seguridad: conexión anónima rechazada, login SCRAM, cuenta read, denegación insert y explicación TLS/X.509/LDAP. Los demás laboratorios forman itinerario de ampliación, no se prometen todos dentro de los 180 minutos.

> Si el formador no tiene Docker listo previamente, el tiempo de descarga y carga de datos puede consumir la sesión. Ejecutar un ensayo completo en el equipo y conservar la salida antes de impartir.

Laboratorio 01 · Preparación de Docker y aislamiento

Ejecución paso a paso

Resultado profesional perseguido

Servicios mongo-indices y mongo-seguridad presentes; comprobar puertos 127.0.0.1.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash. Este paso solo inspecciona contenedores y no exige entrar en mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose ps
```

Precondición que cambia la interpretación

Antes de indexar, asegúrate de que ventas acaba de cargarse y solo contiene *id* como índice.

Evidencia concreta a registrar

Servicios mongo-indices y mongo-seguridad presentes; comprobar puertos 127.0.0.1. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Si Docker no responde, arrancar Docker Desktop antes de cambiar comandos.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 01 · Preparación de Docker y aislamiento

Solución comentada y ampliación

Solución razonada

En docker compose ps distingue nombre del servicio, estado y redirección de puertos. La ausencia de IP antes de 27017 podría publicar en interfaces externas del host: revisa siempre el fichero Compose. La instancia libre es únicamente un experimento local y no contiene datos privados.

Por qué ocurre

Piensa en el registro de una tienda: pedir una factura por numero no debería obligar a abrir cada carpeta. Si existen 200.000 ventas y el filtro devuelve una, la lectura secuencial puede examinar del orden de 200.000 documentos. Esa desproporción es más informativa que un tiempo aislado. El plan puede incluir otras etapas; hay que mirar el árbol entero. Para obtener una línea de base limpia ejecuta la medición ANTES de crear índices secundarios y conserva el tamaño de la colección. Una vez instalado un índice específico, repite exactamente el mismo filtro, proyección y límite.

Qué no permite concluir

Si Docker no responde, arrancar Docker Desktop antes de cambiar comandos. Con solo *id* disponible, el filtro numero o clienteId no dispone de acceso secundario selectivo; normalmente los documentos examinados se acercarán al tamaño completo.

Contraste resuelto con un segundo escenario

```
db.ventas.find({numero:25001}).limit(1).explain("executionStats")
```

La presencia de limit(1) no elimina el recorrido previo de documentos hasta localizar la operación. Compara el número examinado con una consulta por _id y razona qué estructura necesitaría numero.

Ejercicio de transferencia

Describir por qué la instancia sin autenticación no debe ser accesible desde la red.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar servicios mongo-indices y mongo-seguridad presentes; comprobar puertos 127.0.0.1. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 02 · Carga repetible de datos

Ejecución paso a paso

Resultado profesional perseguido

200000 documentos por defecto, o 1000000 con VENTAS; registrar volumen exacto.

Dónde ejecutar y con qué preparación

Primero la terminal para cargar datos. Después entra en mongosh del contenedor mongo-indices para contarlos.

Paso 1 · Terminal del host

```
docker compose exec -T -e VENTAS=200000 mongo-indices mongosh --quiet --file /lab/01_cargar_ventas.js
```

Paso 2 · Shell mongosh

```
use curso_indices
db.ventas.countDocuments({})
```

Precondición que cambia la interpretación

Antes de indexar, asegúrate de que ventas acaba de cargarse y solo contiene *id* como índice.

Evidencia concreta a registrar

200000 documentos por defecto, o 1000000 con VENTAS; registrar volumen exacto. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La carga borra curso_indices.ventas, por eso cada alumno trabaja en entorno local aislado.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 02 · Carga repetible de datos

Solución comentada y ampliación

Solución razonada

Comprueba countDocuments y observa que el índice inicial *id* existe incluso en una colección recién recreada. El generador borra exclusivamente curso_indices.ventas antes de escribir; ejecutarlo de nuevo anula índices y mediciones posteriores. Para un millón de documentos usa VENTAS=1000000 y registra el mayor coste de carga.

Por qué ocurre

En el generador, estado tiene cuatro valores y clienteId recorre 20.000 valores. Para 200.000 ventas caben aproximadamente 50.000 documentos en el primer filtro y diez en el segundo, porque la distribución es determinista. Un índice sobre estado no es necesariamente inútil: puede aportar la ordenación de una consulta compuesta o reducir el trabajo si solo se proyectan campos indexados. Para elegir, compara consultas reales, no etiquetas simplistas como «alta cardinalidad = bueno».

Qué no permite concluir

La carga borra curso_indices.ventas, por eso cada alumno trabaja en entorno local aislado. Con estado de cuatro valores puede ser razonable un escaneo extenso; con numero casi único, leer un tramo corto tiene una ventaja más clara.

Contraste resuelto con un segundo escenario

```
db.ventas.countDocuments({estado:"pagada"})
db.ventas.countDocuments({clienteId:809})
```

La diferencia de órdenes de magnitud entre las dos cuentas muestra qué predicado discrimina más. Repite con un volumen distinto: las proporciones siguen siendo sintéticas y no describen necesariamente una empresa.

Ejercicio de transferencia

Anotar RAM y tiempo de carga para explicar diferencias entre equipos.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar 200000 documentos por defecto, o 1000000 con VENTAS; registrar volumen exacto. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 03 · Inspeccionar estructura y distribución

Ejecución paso a paso

Resultado profesional perseguido

Identificar filtros de alta y baja cardinalidad y tipo BSON de fecha.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.findOne()
db.ventas.distinct("estado")
```

Precondición que cambia la interpretación

Antes de indexar, asegúrate de que ventas acaba de cargarse y solo contiene *id* como índice.

Evidencia concreta a registrar

Identificar filtros de alta y baja cardinalidad y tipo BSON de fecha. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No confundir una distribución determinista docente con una muestra estadística real.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 03 · Inspeccionar estructura y distribución

Solución comentada y ampliación

Solución razonada

Asegúrate de que fecha se representa como Date en MongoDB y no como cadena. Las ventas usan estados de cuatro categorías, regiones de cinco y clientes entre cero y 19999. Una consulta sobre una categoría poco selectiva no desempeña el mismo papel que una búsqueda de un cliente.

Por qué ocurre

En el generador, estado tiene cuatro valores y clienteId recorre 20.000 valores. Para 200.000 ventas caben aproximadamente 50.000 documentos en el primer filtro y diez en el segundo, porque la distribución es determinista. Un índice sobre estado no es necesariamente inútil: puede aportar la ordenación de una consulta compuesta o reducir el trabajo si solo se proyectan campos indexados. Para elegir, compara consultas reales, no etiquetas simplistas como «alta cardinalidad = bueno».

Qué no permite concluir

No confundir una distribución determinista docente con una muestra estadística real. Con estado de cuatro valores puede ser razonable un escaneo extenso; con numero casi único, leer un tramo corto tiene una ventaja más clara.

Contraste resuelto con un segundo escenario

```
db.ventas.countDocuments({estado:"pagada"})
db.ventas.countDocuments({clienteId:809})
```

La diferencia de órdenes de magnitud entre las dos cuentas muestra qué predicado discrimina más. Repite con un volumen distinto: las proporciones siguen siendo sintéticas y no describen necesariamente una empresa.

Ejercicio de transferencia

Proponer una consulta operacional antes de indexar.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar identificar filtros de alta y baja cardinalidad y tipo BSON de fecha. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 04 · Línea base sin índices secundarios

Ejecución paso a paso

Resultado profesional perseguido

Registrar nReturned, totalDocsExamined, totalKeysExamined y COLLSCAN.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({clienteId:809}).explain("executionStats")
```

Precondición que cambia la interpretación

Antes de indexar, asegúrate de que ventas acaba de cargarse y solo contiene *id* como índice.

Evidencia concreta a registrar

Registrar nReturned, totalDocsExamined, totalKeysExamined y COLLSCAN. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** *id* continúa existiendo, aunque no se hayan creado índices secundarios.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 04 · Línea base sin índices secundarios

Solución comentada y ampliación

Solución razonada

Sin índices secundarios el filtro clienteId recorre documentos: habitualmente verás COLLSCAN con muchos docsExamined. La consulta sí devuelve ventas; esa es la prueba positiva de que los datos están. Si aparece IXSCAN antes del experimento, alguien creó un índice o no se recargó la colección.

Por qué ocurre

Piensa en el registro de una tienda: pedir una factura por numero no debería obligar a abrir cada carpeta. Si existen 200.000 ventas y el filtro devuelve una, la lectura secuencial puede examinar del orden de 200.000 documentos. Esa desproporción es más informativa que un tiempo aislado. El plan puede incluir otras etapas; hay que mirar el árbol entero. Para obtener una línea de base limpia ejecuta la medición ANTES de crear índices secundarios y conserva el tamaño de la colección. Una vez instalado un índice específico, repite exactamente el mismo filtro, proyección y límite.

Qué no permite concluir

id continúa existiendo, aunque no se hayan creado índices secundarios. Con solo *id* disponible, el filtro numero o clienteId no dispone de acceso secundario selectivo; normalmente los documentos examinados se acercarán al tamaño completo.

Contraste resuelto con un segundo escenario

```
db.ventas.find({numero:25001}).limit(1).explain("executionStats")
```

La presencia de limit(1) no elimina el recorrido previo de documentos hasta localizar la operación. Compara el número examinado con una consulta por _id y razona qué estructura necesitaría numero.

Ejercicio de transferencia

Ejecutar una segunda vez y discutir por qué cambian milisegundos.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar registrar nReturned, totalDocsExamined, totalKeysExamined y COLLSCAN. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 05 · Búsqueda por número de venta

Ejecución paso a paso

Resultado profesional perseguido

Comparar coste de buscar una clave funcional sin índice.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({numero:25001}).explain("executionStats")
```

Precondición que cambia la interpretación

Antes de indexar, asegúrate de que ventas acaba de cargarse y solo contiene *id* como índice.

Evidencia concreta a registrar

Comparar coste de buscar una clave funcional sin índice. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Elegir número presente según la cantidad de registros cargados.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 05 · Búsqueda por número de venta

Solución comentada y ampliación

Solución razonada

El valor numero=25001 existe para todos los tamaños admitidos. Aunque sea único en el conjunto sintético, eso no convierte a numero en la clave _id. Observa el contador de documentos examinados; después explica qué diferencia hay entre identidad de documento y clave funcional.

Por qué ocurre

Cada colección normal contiene un índice único sobre _id que soporta las búsquedas por identificador del documento. El campo numero no se convierte en _id automáticamente: aunque represente una clave de negocio, es otro camino de acceso. El nombre de la clave funcional importa también por integridad: si numero debe ser irrepetible, un índice unique en numero representa una regla distinta. No elimines ni modifiques _id para demostrar la diferencia.

Qué no permite concluir

Elegir número presente según la cantidad de registros cargados. getIndexes debe contener *id* con unique; verificar que buscar por numero carece de privilegio automático por el mero parecido semántico.

Contraste resuelto con un segundo escenario

```
db.ventas.findOne({_id:db.ventas.findOne()._id})
```

El acceso por _id sirve para identificar un documento; numero es una propiedad distinta. Si la aplicación recibe codigo de negocio, necesitará elegir una clave de acceso apropiada.

Ejercicio de transferencia

Preguntar por qué numero no se beneficia del índice de _id.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comparar coste de buscar una clave funcional sin índice. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 06 · Crear índice simple por cliente

Ejecución paso a paso

Resultado profesional perseguido

Comprobar nombre y campo del índice; evitar una creación duplicada.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({clienteId:1},{name:"ix_cliente"})
db.ventas.getIndexes()
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comprobar nombre y campo del índice; evitar una creación duplicada. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** En la segunda ejecución createIndex del mismo patrón puede reutilizarlo.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 06 · Crear índice simple por cliente

Solución comentada y ampliación

Solución razonada

Al crear ix_cliente se añade una ruta de acceso, no un segundo campo a los documentos. Con getIndexes revisa tanto key como name. Después mide el filtro exacto por cliente; un índice creado sin comprobar su uso constituye una conclusión incompleta.

Por qué ocurre

La clave clienteId identifica a un comprador, no una venta. En 200.000 filas sintéticas cada comprador aparece unas diez veces; por ello no debe crearse como unique. Tras crear ix_cliente, una búsqueda por clienteId debería inspeccionar una fracción pequeña, aunque siga leyendo documentos para devolver el resto de campos. Verifica además que el índice aparece con el nombre correcto en getIndexes() y que se utiliza con el filtro planteado, no con un filtro ajeno.

Qué no permite concluir

En la segunda ejecución createIndex del mismo patrón puede reutilizarlo. Tras crear ix_cliente, el filtro por clienteId debería reducir notablemente totalDocsExamined; el índice no cubre campos de proyección ajenos a su clave.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809}).limit(5).explain("executionStats")
```

No uses una query por fecha para comprobar un índice que solo contiene clienteId. Debe existir correspondencia exacta entre la clave creada y los predicados que quieres estudiar.

Ejercicio de transferencia

Comparar lectura con coste de mantenimiento.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar nombre y campo del índice; evitar una creación duplicada. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 07 · Comparar antes y después

Ejecución paso a paso

Resultado profesional perseguido

Se espera IXSCAN sobre clienteId; totalDocsExamined baja mucho.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({clienteId:809}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Se espera IXSCAN sobre clienteId; totalDocsExamined baja mucho. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El tiempo medido no debe convertirse en una cifra de mejora universal.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 07 · Comparar antes y después

Solución comentada y ampliación

Solución razonada

Para la comparación válida conserva clienteId y todos los demás parámetros. Escribe dos filas con docsExamined y keysExamined, no solo tiempos. El cambio esperado es pasar de explorar la colección a un acceso acotado, aunque el número de documentos devueltos permanezca igual.

Por qué ocurre

nReturned cuenta resultados; totalDocsExamined cuenta exámenes de documentos; totalKeysExamined cuenta entradas de índices leídas. Si salen diez ventas y el motor examinó 200.000 documentos, hay potencial de optimización. Si examina solo diez claves pero diez documentos, la consulta es selectiva pero no cubierta. Si examina cien claves y cero documentos, puede estar cubierta pero seguir recorriendo claves adicionales. Cada contador describe un fenómeno distinto; comparar índices requiere leerlos juntos.

Qué no permite concluir

El tiempo medido no debe convertirse en una cifra de mejora universal. nReturned bajo con docsExamined alto señala trabajo descartado; keysExamined alto con docsExamined cero puede ser consulta cubierta pero aun poco selectiva.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809}).explain("executionStats").executionStats
```

Si nReturned es diez y docsExamined también diez, el acceso ya evita casi todo el escaneo. Un keysExamined de diez tampoco garantiza que el coste total de la petición sea cero.

Ejercicio de transferencia

Justificar qué métrica elegir para un informe técnico.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar se espera IXSCAN sobre clienteId; totalDocsExamined baja mucho. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 08 · Rangos numéricos

Ejecución paso a paso

Resultado profesional perseguido

Repetir explain tras crear índice; observar cuantos valores coinciden.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({importe:{$gte:650,$lt:660}}).explain("executionStats")
db.ventas.createIndex({importe:1})
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Repetir explain tras crear índice; observar cuantos valores coinciden. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Un rango ancho no necesariamente reduce trabajo suficiente.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 08 · Rangos numéricos

Solución comentada y ampliación

Solución razonada

Medir el rango antes de crear {importe:1} muestra el trabajo sin ayuda de esa clave; después se espera reducción de candidatos si el rango es estrecho. Si amplías el rango a casi toda la distribución, el ahorro disminuye; explica por qué el planificador podría preferir otro acceso.

Por qué ocurre

Un rango acota un intervalo: importe entre 650 y 660, por ejemplo. Si el índice empieza por un campo de igualdad y continúa por importe, el motor puede navegar hasta el límite inferior y parar en el superior. En cambio, si la condición selecciona casi todas las ventas, un COLLSCAN puede resultar competitivo. El objeto indexBounds del plan, cuando aparece, ayuda a ver qué intervalo se leyó. No confundas «el índice existe» con «los límites se aprovecharon».

Qué no permite concluir

Un rango ancho no necesariamente reduce trabajo suficiente. El límite inferior o superior de importe aparece en indexBounds si existe clave aprovechable; los documentos devueltos no bastan para medir eficiencia.

Contraste resuelto con un segundo escenario

```
db.ventas.find({importe:{$gte:650,$lt:660}}).explain("executionStats")
```

Un intervalo corto delimita pocos valores. Repite con {$gte:5,$lt:905}: el segundo intervalo contiene casi toda la distribución y la justificación del índice cambia.

Ejercicio de transferencia

Ampliar rango a 50-900 y comparar.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar repetir explain tras crear índice; observar cuantos valores coinciden. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 09 · Filtros de cardinalidad baja

Ejecución paso a paso

Resultado profesional perseguido

Observar plan ganador; podría seleccionar índice o escaneo según coste.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({estado:"pagada"}).explain("executionStats")
db.ventas.createIndex({estado:1})
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Observar plan ganador; podría seleccionar índice o escaneo según coste. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No afirmar que MongoDB está obligado a elegir ix_estado.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 09 · Filtros de cardinalidad baja

Solución comentada y ampliación

Solución razonada

Estado tiene una cardinalidad baja. Un índice simple de estado quizá no sea el acceso más barato si la consulta solicita toda la información de un cuarto de ventas. No fuerces un plan ganador en la respuesta del alumno: anota el que aparezca y contrasta con explain.

Por qué ocurre

En el generador, estado tiene cuatro valores y clienteId recorre 20.000 valores. Para 200.000 ventas caben aproximadamente 50.000 documentos en el primer filtro y diez en el segundo, porque la distribución es determinista. Un índice sobre estado no es necesariamente inútil: puede aportar la ordenación de una consulta compuesta o reducir el trabajo si solo se proyectan campos indexados. Para elegir, compara consultas reales, no etiquetas simplistas como «alta cardinalidad = bueno».

Qué no permite concluir

No afirmar que MongoDB está obligado a elegir ix_estado. Con estado de cuatro valores puede ser razonable un escaneo extenso; con numero casi único, leer un tramo corto tiene una ventaja más clara.

Contraste resuelto con un segundo escenario

```
db.ventas.countDocuments({estado:"pagada"})
db.ventas.countDocuments({clienteId:809})
```

La diferencia de órdenes de magnitud entre las dos cuentas muestra qué predicado discrimina más. Repite con un volumen distinto: las proporciones siguen siendo sintéticas y no describen necesariamente una empresa.

Ejercicio de transferencia

Comparar pagada con consulta más selectiva.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar observar plan ganador; podría seleccionar índice o escaneo según coste. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 10 · Ordenación y TOP N

Ejecución paso a paso

Resultado profesional perseguido

Buscar SORT o recorrido ordenado de índice.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({estado:"pagada"}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Buscar SORT o recorrido ordenado de índice. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Limit no significa ausencia automática de trabajo de ordenación.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 10 · Ordenación y TOP N

Solución comentada y ampliación

Solución razonada

limit(10) afecta a resultados devueltos, pero la ordenación puede requerir trabajo previo. El compuesto estado+fecha permite recorrer fechas recientes dentro de un estado. Comprueba si existe una fase SORT y si el plan elegido aprovecha el orden de la clave.

Por qué ocurre

La consulta «10 pagadas más recientes» combina tres costes: encontrar pagadas, ordenar las coincidencias y devolver diez. Con un compuesto que preserve el orden se puede leer solo el tramo útil; sin él, el motor puede trabajar sobre muchos candidatos antes de entregar diez. Dibuja dos flujos: FILTRO → SORT → LIMIT frente a ACCESO INDEXADO ORDENADO → LIMIT. Una mejora de latencia no prueba por sí sola que se evitó ordenar: hay que comprobar el plan.

Qué no permite concluir

Limit no significa ausencia automática de trabajo de ordenación. Una respuesta de diez resultados puede requerir ordenar cientos de miles de candidatos; el árbol revela si SORT persiste.

Contraste resuelto con un segundo escenario

```
db.ventas.find({estado:"pagada"}).sort({fecha:-1}).limit(1).explain("executionStats")
```

El top 1 es un buen caso para separar ordenación indexada de ordenar un conjunto enorme y después descartar filas. Identifica si la etapa SORT aparece.

Ejercicio de transferencia

Añadir índice {estado:1,fecha:-1} y repetir.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar buscar SORT o recorrido ordenado de índice. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 11 · Compuesto con prefijo útil

Ejecución paso a paso

Resultado profesional perseguido

Confirmar que el patrón satisface el filtro y la ordenación.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({clienteId:1,fecha:-1})
db.ventas.find({clienteId:809}).sort({fecha:-1}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Confirmar que el patrón satisface el filtro y la ordenación. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El índice compuesto tiene claves ordenadas primero por clienteId.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 11 · Compuesto con prefijo útil

Solución comentada y ampliación

Solución razonada

El compuesto agrupa primero clienteId. La consulta que filtra cliente y ordena fecha aprovecha ambos componentes; una consulta global solo por fecha no tiene el mismo prefijo. Verifica que el motor evita una ordenación adicional cuando el plan corresponde al compuesto.

Por qué ocurre

Considera el compuesto {clienteId:1,fecha:-1}: primero agrupa ventas por comprador y solo dentro de cada comprador ordena por fecha reciente. El índice facilita tanto la igualdad por comprador como una consulta que añada fecha, pero no dispone de una ordenación global por fecha que prescinda de clienteId. Es la regla de prefijo: para seleccionar entradas con eficiencia se necesita comenzar por la primera parte de la clave. Distínguela del simple hecho de que una consulta pueda forzar un recorrido de ese índice.

Qué no permite concluir

El índice compuesto tiene claves ordenadas primero por clienteId. El prefijo clienteId delimita la región útil del índice; una consulta solo por fecha no aprovecha el mismo prefijo delimitador.

Contraste resuelto con un segundo escenario

```
db.ventas.find({fecha:{$gte:ISODate("2025-01-01")}}).limit(10).explain("executionStats")
```

La condición anterior no contiene clienteId. Tener fecha como segundo elemento de una clave compuesta no equivale a poseer un índice global que comienza por fecha.

Ejercicio de transferencia

Probar solo fecha y observar la limitación del prefijo.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar confirmar que el patrón satisface el filtro y la ordenación. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 12 · Prefijo inexistente

Ejecución paso a paso

Resultado profesional perseguido

Leer el plan seleccionado; no asumir uso eficiente del índice {clienteId,fecha}.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({fecha:{$gte:ISODate("2025-06-01")}}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Leer el plan seleccionado; no asumir uso eficiente del índice {clienteId,fecha}. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La existencia de otros índices previos puede influir en el plan.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 12 · Prefijo inexistente

Solución comentada y ampliación

Solución razonada

La prueba trata de impedir un error frecuente: «mi índice contiene fecha, luego toda consulta por fecha está optimizada». A falta del prefijo clienteId, el acceso puede ser poco selectivo. Si hay otros índices, explica su influencia y realiza un experimento con hint sin convertirlo en solución de producción.

Por qué ocurre

Considera el compuesto {clienteId:1,fecha:-1}: primero agrupa ventas por comprador y solo dentro de cada comprador ordena por fecha reciente. El índice facilita tanto la igualdad por comprador como una consulta que añada fecha, pero no dispone de una ordenación global por fecha que prescinda de clienteId. Es la regla de prefijo: para seleccionar entradas con eficiencia se necesita comenzar por la primera parte de la clave. Distínguela del simple hecho de que una consulta pueda forzar un recorrido de ese índice.

Qué no permite concluir

La existencia de otros índices previos puede influir en el plan. El prefijo clienteId delimita la región útil del índice; una consulta solo por fecha no aprovecha el mismo prefijo delimitador.

Contraste resuelto con un segundo escenario

```
db.ventas.find({fecha:{$gte:ISODate("2025-01-01")}}).limit(10).explain("executionStats")
```

La condición anterior no contiene clienteId. Tener fecha como segundo elemento de una clave compuesta no equivale a poseer un índice global que comienza por fecha.

Ejercicio de transferencia

Inventariar índices y aislar hipótesis usando hint cuando corresponda.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar leer el plan seleccionado; no asumir uso eficiente del índice {clienteId,fecha}. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 13 · Estrategia ESR paso a paso

Ejecución paso a paso

Resultado profesional perseguido

Comprobar sort y límites del índice en el explain.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({estado:1,fecha:-1,importe:1},{name:"ix_esr"})
db.ventas.find({estado:"pagada",importe:{$gt:500}}).sort({fecha:-1}).limit(20).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comprobar sort y límites del índice en el explain. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** ESR es una heurística: medir claves examinadas, no solo ausencia de SORT.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 13 · Estrategia ESR paso a paso

Solución comentada y ampliación

Solución razonada

ESR no es una fórmula mágica. En {estado,fecha,importe}, el filtro de estado y la ordenación por fecha pueden mejorar la entrega de los primeros resultados. El rango de importe se evalúa dentro de ese orden. Registra cuántas claves se leen hasta encontrar suficientes ventas.

Por qué ocurre

ESR significa Igualdad, Sort, Rango. La clave {estado:1,fecha:-1,importe:1} puede satisfacer estado=pagada, producir fechas ordenadas y evaluar importe. Pero si importe>=890 deja muy pocos resultados, situar importe antes de fecha puede leer muchas menos claves aun a costa de una fase SORT. Ese es el motivo por el que ESR es un punto de partida y no un dogma. El informe debe mostrar keysExamined, docsExamined y la presencia de SORT.

Qué no permite concluir

ESR es una heurística: medir claves examinadas, no solo ausencia de SORT. Si fecha satisface sort y importe es rango posterior, es posible ahorrar SORT aunque se examinen más claves; un ERS selectivo puede resultar mejor.

Contraste resuelto con un segundo escenario

```
db.ventas.find({estado:"pagada",importe:{$gte:890}}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Con rango muy selectivo, una clave estado-importe-fecha puede encontrar pocos registros; con estado-fecha-importe se preserva otra ventaja. La respuesta depende de las estadísticas y de cuántos resultados quieres devolver.

Ejercicio de transferencia

Contrastar ERS en consulta altamente selectiva.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar sort y límites del índice en el explain. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 14 · Estrategia ERS comparada

Ejecución paso a paso

Resultado profesional perseguido

Comparar keysExamined y si necesita SORT.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({estado:1,importe:1,fecha:-1})
db.ventas.find({estado:"pagada",importe:{$gt:850}}).sort({fecha:-1}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comparar keysExamined y si necesita SORT. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Un rango temprano puede mejorar selección pero romper ordenación global.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 14 · Estrategia ERS comparada

Solución comentada y ampliación

Solución razonada

En ERS se prioriza importe como rango después de estado. Con umbral muy alto se puede reducir el tramo leído, pero ya no se obtiene automáticamente un orden global por fecha de todas esas cantidades. Compara ambos diseños sobre idéntico predicado y escribe la razón de la elección.

Por qué ocurre

ESR significa Igualdad, Sort, Rango. La clave {estado:1,fecha:-1,importe:1} puede satisfacer estado=pagada, producir fechas ordenadas y evaluar importe. Pero si importe>=890 deja muy pocos resultados, situar importe antes de fecha puede leer muchas menos claves aun a costa de una fase SORT. Ese es el motivo por el que ESR es un punto de partida y no un dogma. El informe debe mostrar keysExamined, docsExamined y la presencia de SORT.

Qué no permite concluir

Un rango temprano puede mejorar selección pero romper ordenación global. Si fecha satisface sort y importe es rango posterior, es posible ahorrar SORT aunque se examinen más claves; un ERS selectivo puede resultar mejor.

Contraste resuelto con un segundo escenario

```
db.ventas.find({estado:"pagada",importe:{$gte:890}}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Con rango muy selectivo, una clave estado-importe-fecha puede encontrar pocos registros; con estado-fecha-importe se preserva otra ventaja. La respuesta depende de las estadísticas y de cuántos resultados quieres devolver.

Ejercicio de transferencia

Elegir índice ganador justificando coste y frecuencia.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comparar keysExamined y si necesita SORT. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 15 · Covered query consciente

Ejecución paso a paso

Resultado profesional perseguido

Buscar totalDocsExamined=0 si el plan elegido cubre consulta.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({clienteId:809},{_id:0,clienteId:1,fecha:1}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Buscar totalDocsExamined=0 si el plan elegido cubre consulta. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Si el optimizador elige otro índice, probar hint al compuesto pertinente.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 15 · Covered query consciente

Solución comentada y ampliación

Solución razonada

La proyección {_id:0,clienteId:1,fecha:1} busca cobertura con {clienteId,fecha}. Si el motor utiliza la clave correcta y totalDocsExamined es cero, hay evidencia de una consulta cubierta. Agregar descripcion exige recuperar documentos; verifica que esa etapa reaparece.

Por qué ocurre

Una consulta cubierta exige que el índice proporcione tanto la condición como todos los campos devueltos. Con {clienteId:1,fecha:-1}, proyectar exclusivamente clienteId y fecha, excluyendo _id, puede resolverse sin leer documentos. Si agregas descripcion, aparece la necesidad de FETCH. El indicador decisivo es totalDocsExamined=0, junto con un plan compatible; nReturned puede ser mayor que cero. Una consulta cubierta no implica que solo se examinen las claves necesarias para los resultados.

Qué no permite concluir

Si el optimizador elige otro índice, probar hint al compuesto pertinente. Excluir _id y devolver exclusivamente clienteId y fecha permite, si se elige el compuesto, totalDocsExamined=0; agregar descripcion rompe la cobertura.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809},{_id:0,clienteId:1,fecha:1}).explain("executionStats")
```

En una consulta cubierta la lectura de documentos puede quedar en cero. Añade descripcion a la proyección y contrasta con el caso anterior sin cambiar filtro.

Ejercicio de transferencia

Añadir descripcion a proyección y observar FETCH.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar buscar totalDocsExamined=0 si el plan elegido cubre consulta. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 16 · Explicar el árbol de ejecución

Ejecución paso a paso

Resultado profesional perseguido

Distinguir winningPlan, inputStages e indexName según motor de ejecución.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
const e=db.ventas.find({clienteId:809}).explain("executionStats");
printjson({queryPlanner:e.queryPlanner,stats:e.executionStats})
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Distinguir winningPlan, inputStages e indexName según motor de ejecución. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Las representaciones de stage pueden variar con SBE y versión.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 16 · Explicar el árbol de ejecución

Solución comentada y ampliación

Solución razonada

Separa plan ganador y estadísticas. Una estructura SBE puede mostrar el índice en rutas distintas a las de planes clásicos. La lectura razonada identifica indexName, FILTER/IXSCAN/FETCH/SORT si aparecen y luego los contadores. Copiar un JSON entero sin interpretación no acredita el objetivo.

Por qué ocurre

COLLSCAN y IXSCAN nombran recorridos; FETCH señala recuperación de documentos. En explain puede verse FETCH encima de IXSCAN: significa que el índice orienta el acceso y los documentos se leen para terminar de resolver filtro o proyección. Un plan no es una cadena plana: puede haber ramas y etapas diferentes, especialmente con el motor de ejecución SBE de versiones modernas. Enseña a localizar indexName y los contadores, no a memorizar una captura específica de otro lanzamiento.

Qué no permite concluir

Las representaciones de stage pueden variar con SBE y versión. En un plan INDEX_SCAN+FETCH, las claves examinadas no deben confundirse con documentos leídos: ambas cantidades contestan preguntas distintas.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809,estado:"pagada"}).explain("executionStats")
```

La presencia de IXSCAN no implica que el filtro estado esté también resuelto por el mismo índice. Un FETCH puede aplicar parte del filtro sobre documentos recuperados.

Ejercicio de transferencia

Localizar todas las subetapas que recuperan documentos.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar distinguir winningPlan, inputStages e indexName según motor de ejecución. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 17 · Experimentar con hint

Ejecución paso a paso

Resultado profesional perseguido

Comparar usando un índice fijo.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({clienteId:809}).hint({clienteId:1}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comparar usando un índice fijo. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** hint es diagnóstico controlado, no remedio permanente.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 17 · Experimentar con hint

Solución comentada y ampliación

Solución razonada

Con hint, una clave elegida por el usuario sustituye el criterio del planificador para el experimento. El mismo filtro puede rendir peor con un índice menos selectivo. Anota la diferencia y explica por qué el código de aplicación normalmente no debería perpetuar ese hint.

Por qué ocurre

hint resulta útil para comparar un candidato con otro sin depender de la decisión del planificador. Solo tiene sentido si el índice existe y el filtro puede interpretarse adecuadamente. Forzar un acceso malo puede elevar muchísimo el trabajo. Enseña a retirar hint antes de trasladar una consulta a código de aplicación: una pista fija que hoy funciona puede bloquear mejoras después de que cambien los datos, el motor o la distribución.

Qué no permite concluir

hint es diagnóstico controlado, no remedio permanente. Forzar clienteId:1 solo tiene sentido cuando la clave existe; explica si el plan impuesto coincide con la elección espontánea del optimizador.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809}).hint({clienteId:1}).explain("executionStats")
```

Esta orden fija el camino de acceso y permite contrastarlo con la selección espontánea. Si el optimizador elige otro plan, discute por qué las condiciones de la consulta lo favorecen.

Ejercicio de transferencia

Registrar cuándo el optimizador habría elegido otra ruta.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comparar usando un índice fijo. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 18 · Multikey con etiquetas

Ejecución paso a paso

Resultado profesional perseguido

Observar isMultiKey cuando aparezca y entradas del índice.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({etiquetas:1},{name:"ix_etiquetas"})
db.ventas.find({etiquetas:"urgente"}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Observar isMultiKey cuando aparezca y entradas del índice. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La indexación multikey ocurre automáticamente con arrays.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 18 · Multikey con etiquetas

Solución comentada y ampliación

Solución razonada

El índice de etiquetas se vuelve multikey al indexar un array. Aunque una venta tenga varias etiquetas, MongoDB devuelve un documento una sola vez para una coincidencia. Analiza la diferencia entre entradas del índice y documentos de salida, y comprueba isMultiKey si la representación lo muestra.

Por qué ocurre

El campo etiquetas guarda varios valores; al indexarlo, MongoDB crea entradas asociadas a cada valor distinto del array. Un documento etiquetado como venta, web y urgente puede originar varias claves, pero el resultado lógico de find no debe aparecer triplicado. Observa isMultiKey donde esté disponible. El coste no depende solo de cuántos documentos hay, también de cuántos elementos contienen sus arrays. Una actualización de etiquetas puede modificar varias entradas.

Qué no permite concluir

La indexación multikey ocurre automáticamente con arrays. Un índice sobre etiquetas genera entradas para elementos del array; observar isMultiKey y confirmar que una venta coincidente se devuelve una sola vez.

Contraste resuelto con un segundo escenario

```
db.ventas.find({etiquetas:"web"}).limit(10).explain("executionStats")
```

La coincidencia de una etiqueta no debe duplicar una venta aunque el array tenga varias etiquetas. Revisa el documento real y diferencia los elementos del array de las entradas del índice.

Ejercicio de transferencia

Analizar por qué un documento con dos etiquetas no se duplica en la salida.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar observar isMultiKey cuando aparezca y entradas del índice. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 19 · Multikey y arrays anidados

Ejecución paso a paso

Resultado profesional perseguido

Comprobar índice compuesto con un array y un escalar.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.multikey_demo.insertMany([{a:[1,2],b:"X"},{a:[2,3],b:"Y"}])
db.multikey_demo.createIndex({a:1,b:1})
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comprobar índice compuesto con un array y un escalar. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No convertir el laboratorio en permiso para múltiples campos array simultáneos.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 19 · Multikey y arrays anidados

Solución comentada y ampliación

Solución razonada

Un array junto a un escalar es un escenario válido para un compuesto multikey. Dos campos array en el mismo documento bajo una clave compuesta pueden disparar la limitación de parallel arrays. El segundo intento es una prueba negativa deliberada y debe realizarse en la colección de demostración.

Por qué ocurre

Un índice compuesto puede combinar un array y un campo escalar, por ejemplo {etiquetas:1,estado:1}. Sin embargo, los índices compuestos multikey tienen restricciones si hay arrays paralelos en varios campos del mismo documento. No construyas un ejercicio que falle sin explicarlo: crea antes un documento con a:[1,2] y b:"X"; después ensaya de forma aislada un segundo documento donde a y b sean arrays y captura el error. La restricción afecta al modelo de documentos admitido por ese índice.

Qué no permite concluir

No convertir el laboratorio en permiso para múltiples campos array simultáneos. Un compuesto puede tener un campo array y otros escalares; múltiples campos array del mismo documento pueden activar la limitación parallel arrays.

Contraste resuelto con un segundo escenario

```
db.multikey_demo.insertOne({a:[1,2],b:["X","Y"]})
```

Con un compuesto sobre a y b, este intento puede provocar error de arrays paralelos. Es una prueba negativa de esquema; debe realizarse en colección separada y capturar el error, no romper ventas.

Ejercicio de transferencia

Intentar un modelo con a y b arrays en otro documento y capturar el error.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar índice compuesto con un array y un escalar. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 20 · Búsqueda textual clásica

Ejecución paso a paso

Resultado profesional perseguido

Localizar descripciones y distinguir `$text` de regex.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({descripcion:"text"},{name:"ix_descripcion_texto",default_language:"spanish"})
db.ventas.find({$text:{$search:"bicicleta"}}).limit(5)
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Localizar descripciones y distinguir `$text` de regex. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Una sola colección acepta restricciones específicas de índices text clásicos.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 20 · Búsqueda textual clásica

Solución comentada y ampliación

Solución razonada

La búsqueda $text requiere el índice de texto que la soporta; un like SQL no es su equivalente. Comprueba que devuelve documentos con términos coincidentes y registra qué campo se indexó. Un texto de búsqueda ausente puede producir cero resultados sin significar un error de índice.

Por qué ocurre

El índice text clásico crea una representación de términos, no de cualquier subcadena. Una búsqueda de «bicicleta» sobre descripcion se parece a buscar palabras en un índice invertido. Stemming y reglas lingüísticas pueden variar por default_language; no esperes que coincida siempre un fragmento a mitad de palabra. Una colección autogestionada dispone de las limitaciones propias del índice text; Atlas Search es una tecnología diferente y no se obtiene automáticamente por usar $text.

Qué no permite concluir

Una sola colección acepta restricciones específicas de índices text clásicos. La búsqueda $text requiere un índice de tipo text en esta colección; puede aplicar stemming según idioma y no equivale a buscar subcadenas.

Contraste resuelto con un segundo escenario

```
db.ventas.find({$text:{$search:"cafe"}},{_id:0,descripcion:1}).limit(5)
```

La búsqueda por término devuelve descripciones que contienen vocabulario relacionado según el analizador, no todos los textos que incluyen literalmente una cadena de caracteres arbitraria.

Ejercicio de transferencia

Probar palabras con y sin tildes y comentar stemming.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar localizar descripciones y distinguir `$text` de regex. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 21 · Relevancia por textScore

Ejecución paso a paso

Resultado profesional perseguido

Comparar ranking y términos de búsqueda.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({$text:{$search:"bicicleta cafe"}},{_id:0,descripcion:1,puntuacion:
{$meta:"textScore"}}).sort({puntuacion:{$meta:"textScore"}}).limit(5)
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comparar ranking y términos de búsqueda. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El score no es probabilidad absoluta ni ranking de Atlas Search.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 21 · Relevancia por textScore

Solución comentada y ampliación

Solución razonada

textScore funciona como criterio de ranking dentro del resultado $text actual. No compares scores de consultas distintas como probabilidades absolutas. Introduce dos documentos de demo con distinta presencia de términos y explica qué evidencia respalda una variación del orden.

Por qué ocurre

textScore ofrece una puntuación de relevancia para coincidencias de $text. No es una probabilidad, ni una distancia, ni equivale a la puntuación de Atlas Search. Si cambias el texto de búsqueda cambias también el contexto en que se interpreta el score. Proyecta la puntuación con {$meta:"textScore"} y ordénala con el mismo metadato. Compara dos descripciones, una con un término y otra con varios, sin prometer un valor numérico concreto.

Qué no permite concluir

El score no es probabilidad absoluta ni ranking de Atlas Search. El resultado de $meta textScore solo es comparable dentro de la búsqueda realizada; ordenar por score no sustituye un índice ordenado convencional.

Contraste resuelto con un segundo escenario

```
db.ventas.find({$text:{$search:"cafe bicicleta"}},{descripcion:1,p:{$meta:"textScore"}}).sort({p:
{$meta:"textScore"}}).limit(3)
```

La relevancia se interpreta dentro de esta consulta y de estos términos. El score sirve para ordenar, pero no puede expresarse como probabilidad de que el documento sea «correcto».

Ejercicio de transferencia

Introducir descripción con repetición de términos y estudiar relevancia.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comparar ranking y términos de búsqueda. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 22 · Geoespacial 2dsphere

Ejecución paso a paso

Resultado profesional perseguido

Validar que se recuperan puntos en radio dado.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({ubicacion:"2dsphere"},{name:"ix_ubicacion"})
db.ventas.find({ubicacion:{$near:{$geometry:{type:"Point",coordinates:[-3.70,40.42]},$maxDistance:1000}}}).limit(5)
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Validar que se recuperan puntos en radio dado. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El orden GeoJSON es longitud antes que latitud.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 22 · Geoespacial 2dsphere

Solución comentada y ampliación

Solución razonada

El índice 2dsphere permite la búsqueda $near por ubicación GeoJSON. Confirma que coordinates lleva [longitud,latitud] y que $maxDistance se interpreta en metros. El contador de resultados puede cambiar al ampliar radio; no lo confundas con un cambio en la calidad del índice.

Por qué ocurre

GeoJSON representa un punto como {type:"Point", coordinates:[longitud,latitud]}. Invertir ese orden es un error frecuente porque muchas interfaces enseñan primero latitud. Un índice 2dsphere interpreta geometría sobre la superficie terrestre y habilita ciertas consultas de proximidad. La validez geográfica y la exactitud de las coordenadas son parte del experimento: no basta con que la estructura JSON sea sintácticamente correcta. Usa puntos sintéticos, no geolocalización privada.

Qué no permite concluir

El orden GeoJSON es longitud antes que latitud. 2dsphere acepta puntos GeoJSON válidos; la latitud y longitud intercambiadas cambian el lugar geográfico, no son un detalle tipográfico.

Contraste resuelto con un segundo escenario

```
db.ventas.findOne({}, {ubicacion:1,_id:0})
```

El orden de las coordenadas del punto es [longitud,latitud]. Verificar el dato antes de crear el índice evita atribuir a MongoDB un error que en realidad está en la geometría.

Ejercicio de transferencia

Cambiar radio de 200 a 2000 metros.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar validar que se recuperan puntos en radio dado. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 23 · Índice hashed

Ejecución paso a paso

Resultado profesional perseguido

Comprobar accesos de igualdad y limitación conceptual de rangos.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({idGrupo:"hashed"},{name:"ix_grupo_hash"})
db.ventas.find({idGrupo:123}).explain("executionStats")
```

Precondición que cambia la interpretación

Los índices ya construidos pueden influir en el plan ganador; comprueba getIndexes antes de inferir la causa.

Evidencia concreta a registrar

Comprobar accesos de igualdad y limitación conceptual de rangos. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Una clave hashed no preserva orden del valor original.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 23 · Índice hashed

Solución comentada y ampliación

Solución razonada

Hashed apoya igualdad pero no conserva el orden numérico que un rango requiere. Compara find({idGrupo: 123}) con una búsqueda de 100 a 200 y justifica por qué un árbol de hashes no es un índice ordenado de idGrupo.

Por qué ocurre

Una clave hashed registra un hash del valor; favorece ciertos escenarios de distribución, especialmente relacionados con sharding. Como el hash destruye el orden natural, no sirve para resolver rangos de idGrupo como lo haría un índice ascendente. Una consulta de igualdad sí puede utilizarlo. No presentes hashed como «índice más rápido» ni como función de seguridad criptográfica de contraseñas: resuelve un problema de distribución y acceso por igualdad.

Qué no permite concluir

Una clave hashed no preserva orden del valor original. El hash dispersa valores con proximidad numérica; una comparación por rango no conserva el orden y no puede sacar la misma ventaja.

Contraste resuelto con un segundo escenario

```
db.ventas.find({idGrupo:{$gte:100,$lte:200}}).explain("executionStats")
```

El índice hashed sirve para igualdad, pero los hashes de 100 y 200 no delimitan una secuencia ordenada de los valores intermedios.

Ejercicio de transferencia

Probar idGrupo entre 100 y 200 sin prometer aceleración.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar accesos de igualdad y limitación conceptual de rangos. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 24 · Unicidad sin interferir con ventas

Ejecución paso a paso

Resultado profesional perseguido

Comprobar que el primer insert funciona y el índice es unique.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.facturas_demo.drop()
db.facturas_demo.createIndex({codigo:1},{unique:true})
db.facturas_demo.insertOne({codigo:"F-001"})
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Comprobar que el primer insert funciona y el índice es unique. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No usar la colección de ventas para ensayos destructivos de unicidad.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 24 · Unicidad sin interferir con ventas

Solución comentada y ampliación

Solución razonada

El índice único se crea antes de insertar en facturas_demo. Comprueba que la primera factura entra y que la segunda inserción duplicada falla. La colección ventas no debe alterarse para demostrar un error de integridad de facturas.

Por qué ocurre

unique convierte la integridad de datos en una regla atómica del servidor. Si ya existe codigo F-001, intentar insertar otra factura con el mismo valor debe fallar con E11000. No intentes observar el error insertando datos duplicados en ventas, porque perjudicarías los demás laboratorios. Usa facturas_demo. Antes de construir unique sobre datos existentes hay que detectar duplicados y decidir cómo resolverlos. La unicidad tampoco sustituye una política de autorización.

Qué no permite concluir

No usar la colección de ventas para ensayos destructivos de unicidad. El intento de crear clave duplicada bajo unique debe producir E11000. El alumno debe comprobar que la segunda operación no insertó el documento.

Contraste resuelto con un segundo escenario

```
try {db.facturas_demo.insertOne({codigo:"F-001"})} catch(e) {print(e.code)}
```

El conflicto 11000 es intencional si F-001 ya existe bajo la restricción. Si no falla, revisa primero que la colección y el índice unique se prepararon como estaba indicado.

Ejercicio de transferencia

Intentar un segundo F-001 dentro de try/catch.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar que el primer insert funciona y el índice es unique. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 25 · Doble clave y error E11000

Ejecución paso a paso

Resultado profesional perseguido

Identificar el código 11000 como integridad, no conectividad.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
try { db.facturas_demo.insertOne({codigo:"F-001"}); } catch(e) {print(e.code)}
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Identificar el código 11000 como integridad, no conectividad. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La restricción se aplica en el servidor incluso si el cliente no valida.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 25 · Doble clave y error E11000

Solución comentada y ampliación

Solución razonada

E11000 confirma que la unicidad se verificó en el servidor. Captura el código, no solo el texto del mensaje, que puede variar. Después verifica countDocuments({codigo:"F-001"}) para demostrar que no hubo una segunda factura insertada.

Por qué ocurre

unique convierte la integridad de datos en una regla atómica del servidor. Si ya existe codigo F-001, intentar insertar otra factura con el mismo valor debe fallar con E11000. No intentes observar el error insertando datos duplicados en ventas, porque perjudicarías los demás laboratorios. Usa facturas_demo. Antes de construir unique sobre datos existentes hay que detectar duplicados y decidir cómo resolverlos. La unicidad tampoco sustituye una política de autorización.

Qué no permite concluir

La restricción se aplica en el servidor incluso si el cliente no valida. El intento de crear clave duplicada bajo unique debe producir E11000. El alumno debe comprobar que la segunda operación no insertó el documento.

Contraste resuelto con un segundo escenario

```
try {db.facturas_demo.insertOne({codigo:"F-001"})} catch(e) {print(e.code)}
```

El conflicto 11000 es intencional si F-001 ya existe bajo la restricción. Si no falla, revisa primero que la colección y el índice unique se prepararon como estaba indicado.

Ejercicio de transferencia

Crear unique compuesto por tienda y referencia.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar identificar el código 11000 como integridad, no conectividad. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 26 · Índices parciales

Ejecución paso a paso

Resultado profesional perseguido

Revisar si el filtro de la consulta implica el filtro parcial.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.createIndex({clienteId:1},{name:"ix_cliente_pagada",partialFilterExpression:{estado:"pagada"}})
db.ventas.find({clienteId:809,estado:"pagada"}).explain("executionStats")
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Revisar si el filtro de la consulta implica el filtro parcial. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El optimizador no puede usar ese índice para todas las ventas de un cliente.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 26 · Índices parciales

Solución comentada y ampliación

Solución razonada

La consulta que incluye estado=pagada es compatible con el filtro parcial, mientras que la consulta sin estado no queda representada completamente en ese índice. La prueba demuestra corrección además de reducción de tamaño: nunca optimizar si ello puede excluir resultados válidos.

Por qué ocurre

Un índice partial almacena solo documentos que cumplen partialFilterExpression. Una búsqueda puede aprovecharlo si sus condiciones garantizan que todos los resultados están dentro de esa fracción. Si la consulta no exige estado=pagada, utilizar un índice que solo contiene pagadas perdería datos: el planificador no debe considerarlo suficiente. Sparse es un mecanismo diferente, basado en presencia del campo. Diseña la regla a partir del patrón de consultas, no solo del tamaño deseado.

Qué no permite concluir

El optimizador no puede usar ese índice para todas las ventas de un cliente. El plan puede emplear el índice parcial si la consulta restringe estado=pagada; si omite estado, ese índice no representa el conjunto completo.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809,estado:"pagada"}).explain("executionStats")
```

Para utilizar la versión parcial de un índice por cliente, el predicado debe garantizar pertenencia al subconjunto pagado. El planificador evita resultados incompletos.

Ejercicio de transferencia

Comparar sin filtro de estado.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar revisar si el filtro de la consulta implica el filtro parcial. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 27 · TTL y fechas BSON

Ejecución paso a paso

Resultado profesional perseguido

Verificar que creadoEn sea BSON Date.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.sesiones_demo.drop()
db.sesiones_demo.createIndex({creadoEn:1},{expireAfterSeconds:60})
db.sesiones_demo.insertOne({tipo:"caducado",creadoEn:new Date(Date.now()-120000)})
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Verificar que creadoEn sea BSON Date. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El monitor TTL trabaja periódicamente; la eliminación puede tardar.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 27 · TTL y fechas BSON

Solución comentada y ampliación

Solución razonada

La marca creadoEn debe ser BSON Date. El documento creado hace dos minutos ya es elegible con TTL de 60 segundos, pero la eliminación no tiene instante exacto. Comprueba primero que fue insertado y, tras esperar, inspecciona el monitor mediante las observaciones repetidas.

Por qué ocurre

TTL automatiza eliminación eventual de documentos expirados, como tokens o sesiones de demostración. No es una agenda exacta al segundo: un monitor en segundo plano selecciona documentos elegibles y puede retrasarse. Un índice TTL convencional es de un solo campo sobre fechas BSON; no utilices strings que parezcan fechas. Nunca apliques TTL accidentalmente a ventas que deban conservarse por obligación legal. La política de retención es una decisión de negocio y de cumplimiento, no un truco de rendimiento.

Qué no permite concluir

El monitor TTL trabaja periódicamente; la eliminación puede tardar. El monitor TTL elimina documentos elegibles en segundo plano. La observación correcta es eventual, no una promesa de borrado al cumplir exacto sesenta segundos.

Contraste resuelto con un segundo escenario

```
db.sesiones_demo.getIndexes()
```

Comprueba que expireAfterSeconds pertenece al índice apropiado y que el campo almacena fechas. La eliminación es por caducidad, no una orden directa de deleteMany.

Ejercicio de transferencia

Consultar countDocuments y repetir transcurrido el intervalo.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar verificar que creadoEn sea BSON Date. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 28 · TTL: conservar documento reciente

Ejecución paso a paso

Resultado profesional perseguido

Verificar que un documento reciente permanece al principio.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.sesiones_demo.insertOne({tipo:"reciente",creadoEn:new Date()})
db.sesiones_demo.find().toArray()
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Verificar que un documento reciente permanece al principio. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La misma política TTL puede dar distintos tiempos observables según carga.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 28 · TTL: conservar documento reciente

Solución comentada y ampliación

Solución razonada

Una sesión reciente puede permanecer hasta superar la edad fijada y hasta que el monitor la procese. No interpretar su presencia inmediata como fallo. Deja pasar un intervalo y compara los identificadores; el programa no debe prometer que desaparece en un segundo determinado.

Por qué ocurre

expireAfterSeconds:60 se interpreta con respecto al valor del campo Date: un documento creado hace dos minutos ya es elegible, uno creado ahora no. El monitor puede tardar en ejecutar el borrado. Si se usa expireAfterSeconds:0, el propio campo suele representar el instante de expiración. La diferencia es crucial: no mezclar el campo fechaDeCreacion con fechaDeCaducidad sin ajustar el valor. Una consulta countDocuments demuestra el estado observado, no el instante exacto del monitor.

Qué no permite concluir

La misma política TTL puede dar distintos tiempos observables según carga. Una fecha dos minutos anterior con TTL de un minuto ya cumple el criterio; una fecha actual aún no, aunque ambas existan al insertar.

Contraste resuelto con un segundo escenario

```
db.sesiones_demo.find({}, {_id:0,creadoEn:1,tipo:1}).toArray()
```

Distingue elegibilidad temporal de ejecución efectiva del monitor. La misma observación repetida más tarde constituye la prueba razonable de TTL.

Ejercicio de transferencia

Diseñar criterio de limpieza sin prometer caducidad instantánea.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar verificar que un documento reciente permanece al principio. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 29 · Ocultar índice sin borrarlo

Ejecución paso a paso

Resultado profesional perseguido

Inspeccionar hidden:true y repetir explain de cliente.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.hideIndex("ix_cliente")
db.ventas.getIndexes().filter(x=>x.name==="ix_cliente")
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Inspeccionar hidden:true y repetir explain de cliente. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El índice oculto sigue ocupando espacio y coste de mantenimiento.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 29 · Ocultar índice sin borrarlo

Solución comentada y ampliación

Solución razonada

hideIndex no suprime datos del catálogo ni ahorra escrituras. Si el índice oculto era útil, el plan cambia al repetir una consulta de cliente. Deshaz el experimento con unhideIndex y vuelve a comprobar hidden para no dejar el entorno en un estado sorpresivo.

Por qué ocurre

hideIndex oculta un índice al planificador sin retirarlo físicamente. Permite estudiar el efecto que tendría su ausencia y después revertir la prueba; mientras esté oculto sigue consumiendo espacio y trabajo en escrituras. dropIndex sí elimina el índice. Evita modificar *id* y documenta la reversión. Esta herramienta es útil para retirada prudente, pero la prueba de un día no captura consultas mensuales o cierres contables.

Qué no permite concluir

El índice oculto sigue ocupando espacio y coste de mantenimiento. hideIndex vuelve invisible el índice al planificador pero mantiene su actualización. Después de la prueba conviene restaurarlo con unhideIndex.

Contraste resuelto con un segundo escenario

```
db.ventas.getIndexes().filter(x=>x.hidden)
```

Esta consulta permite saber qué índice fue ocultado. Restaurarlo antes de continuar evita comparar planes de un estado desconocido.

Ejercicio de transferencia

Restaurar con unhideIndex para no dejar la práctica alterada.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar inspeccionar hidden:true y repetir explain de cliente. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 30 · Estadísticas de uso

Ejecución paso a paso

Resultado profesional perseguido

Observar operaciones de uso del índice desde las estadísticas disponibles.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.aggregate([{$indexStats:{}}]).toArray()
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Observar operaciones de uso del índice desde las estadísticas disponibles. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Poco uso durante laboratorio no indica inutilidad en producción.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 30 · Estadísticas de uso

Solución comentada y ampliación

Solución razonada

$indexStats depende del período y de lo que realmente se consultó. Una clave con cero accesos durante una sesión corta no es necesariamente redundante. Identifica índices usados por los scripts y redacta un calendario de revisión que incluya ciclos de negocio.

Por qué ocurre

$indexStats informa de accesos observados desde un período de medición, no de la utilidad eterna del índice. Una baja cifra puede deberse a que la función que lo utiliza solo corre una vez al mes. Complementa uso, patrones de consultas, tamaños e impacto en escrituras. Para un inventario técnico, anota nombre, clave, opciones, consultas servidas y motivo de conservación. La retirada de índices debe planificarse y monitorizarse.

Qué no permite concluir

Poco uso durante laboratorio no indica inutilidad en producción. $indexStats cuenta accesos desde su ventana de registro; un índice marcado poco usado durante diez minutos puede ser esencial en el cierre mensual.

Contraste resuelto con un segundo escenario

```
db.ventas.aggregate([{$indexStats:{}}]).toArray()
```

Los contadores solo describen la ventana observada; el índice de una tarea trimestral puede no tener accesos durante un curso de tres horas.

Ejercicio de transferencia

Enumerar un ciclo de auditoría de índices de 30 días.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar observar operaciones de uso del índice desde las estadísticas disponibles. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 31 · Tamaño e inventario final

Ejecución paso a paso

Resultado profesional perseguido

Interpretar dataSize, storageSize e indexSizes con prudencia.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.stats()
db.ventas.getIndexes()
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Interpretar dataSize, storageSize e indexSizes con prudencia. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Los tamaños físicos dependen de compresión, almacenamiento y volumen.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 31 · Tamaño e inventario final

Solución comentada y ampliación

Solución razonada

indexSizes mide bytes de las estructuras; no equivale a documentos adicionales ni a RAM garantizada. Relaciona el inventario con tres consultas que representa el negocio y propon qué dejarías, qué ocultarías provisionalmente y qué no construirías.

Por qué ocurre

La memoria cache almacena páginas de datos e índices; cuanto más voluminosas sean las estructuras activas, mayor puede ser la presión sobre recursos. indexSizes y stats aportan tamaños físicos, pero compresión y patrones de acceso impiden inferir linealmente el rendimiento. Un índice parcial puede ser útil si su subconjunto representa una parte pequeña de datos. La pregunta no es «¿cuántos índices puedo crear?», sino «¿qué conjunto mínimo justifica cada coste?».

Qué no permite concluir

Los tamaños físicos dependen de compresión, almacenamiento y volumen. stats e indexSizes permiten estudiar memoria y espacio, pero compresión y working set impiden convertir cada byte en una predicción de latencia.

Contraste resuelto con un segundo escenario

```
db.ventas.stats().indexSizes
```

El coste de bytes se pone en relación con los patrones de consulta y con el costo de mantener cada índice en escrituras.

Ejercicio de transferencia

Defender qué índices conservar para tres consultas prioritarias.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar interpretar dataSize, storageSize e indexSizes con prudencia. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 32 · Caso integrador de diagnóstico

Ejecución paso a paso

Resultado profesional perseguido

Redactar diagnóstico con filtro, sort, índice, nReturned, examined y mejora propuesta.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-indices, sobre curso_indices; no en PowerShell.

Paso 1 · Abrir el shell del contenedor

```
docker compose exec -it mongo-indices mongosh
```

Paso 2 · Seleccionar base y ejecutar

```
use curso_indices
db.ventas.find({clienteId:809,estado:"pagada",importe:{$gte:700}}).sort({fecha:-1}).limit(10).explain("executionStats")
```

Precondición que cambia la interpretación

Las operaciones de integridad y caducidad se realizan sobre colecciones sintéticas independientes para no alterar ventas.

Evidencia concreta a registrar

Redactar diagnóstico con filtro, sort, índice, nReturned, examined y mejora propuesta. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No crear diez índices nuevos para una consulta ocasional.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 32 · Caso integrador de diagnóstico

Solución comentada y ampliación

Solución razonada

El caso de cierre exige presentar consulta, plan, contadores, candidato de índice y coste añadido. Una respuesta de buena calidad explica también una limitación: el índice elegido puede beneficiar esta consulta pero perjudicar escrituras u otras ordenaciones.

Por qué ocurre

La propuesta final conecta consultas frecuentes con índices mantenidos. Ejemplo: cliente → {clienteId:1}; pagadas por fecha → {estado:1,fecha:-1}; búsqueda textual → {descripcion:"text"}. Justifica por qué otras necesidades son excepcionales o se atienden de otra forma. Añadir índices sin uso aumenta superficie de mantenimiento. El alumno debe defender tanto lo que crea como lo que decide no crear; es una decisión de arquitectura de acceso a datos.

Qué no permite concluir

No crear diez índices nuevos para una consulta ocasional. La solución final debe mapear cada índice conservado a uno o varios patrones frecuentes y justificar índices descartados por coste de escritura.

Contraste resuelto con un segundo escenario

```
db.ventas.find({clienteId:809,estado:"pagada"}).sort({fecha:-1}).limit(10).explain("executionStats")
```

En el diagnóstico final el alumno debe conservar una tabla con dos diseños alternativos y defender uno; no basta con afirmar que la consulta ahora «va más rápida».

Ejercicio de transferencia

Diseñar A/B de dos candidatos y justificar con evidencia.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar redactar diagnóstico con filtro, sort, índice, nReturned, examined y mejora propuesta. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir